EXAMPLE DRILLS PROJECT · 2 × · 95 MIN

Paged KV-cache drills (example)

Two one-hour drills that rebuild the heart of vLLM v1's memory management and scheduling on CPU: a ref-counted block pool with prefix caching, and one step of a continuous-batching scheduler with chunked prefill.

How to use this project

Each drill implements one core mechanism of an LLM serving engine in plain Python. Everything runs on CPU in well under a second.

```
python3 -m venv .venv && .venv/bin/pip install pytest
./check.sh a1                 # test your mine/a1_block_pool.py
./check.sh a1 --solutions     # test the reference solution
./check.sh all                # every drill
```

1. **Read the knowledge page first** (10 minutes). It explains why paged KV cache and continuous batching exist, with diagrams linked to the vLLM source.
2. **Read the drill's explainer page**: the problem, an analogy, a diagram and a step-through animation.
3. **Implement it** in `mine/` within the time on the card. Only open the hints when you have been stuck for 10—15 minutes: L1 points the direction, L2 breaks the problem down, L3 is almost the answer.
4. **Run the tests**, fix, then compare with the commented reference solution and the follow-up questions.

Knowledge

Paged KV cache and continuous batching

Why an LLM server stores the KV cache in fixed-size blocks, how blocks become a prefix cache, and how a scheduler decides which tokens run in each step.

Drills

Track A · Memory and scheduling

The two data structures every LLM serving engine is built around.

☐ **A1** 50 min · medium

BlockPool: ref counts, LRU free queue, prefix cache

explainer & hints mine/ tests/ solutions/

☐ **A2** 45 min · medium

Scheduler step: token budget, chunked prefill, FCFS admission

explainer & hints mine/ tests/ solutions/

PDF book: paged-kv-drills.pdf

KNOWLEDGE

Paged KV cache and continuous batching

Why an LLM server stores the KV cache in fixed-size blocks, how blocks become a prefix cache, and how a scheduler decides which tokens run in each step.

The problem: KV memory is the bottleneck

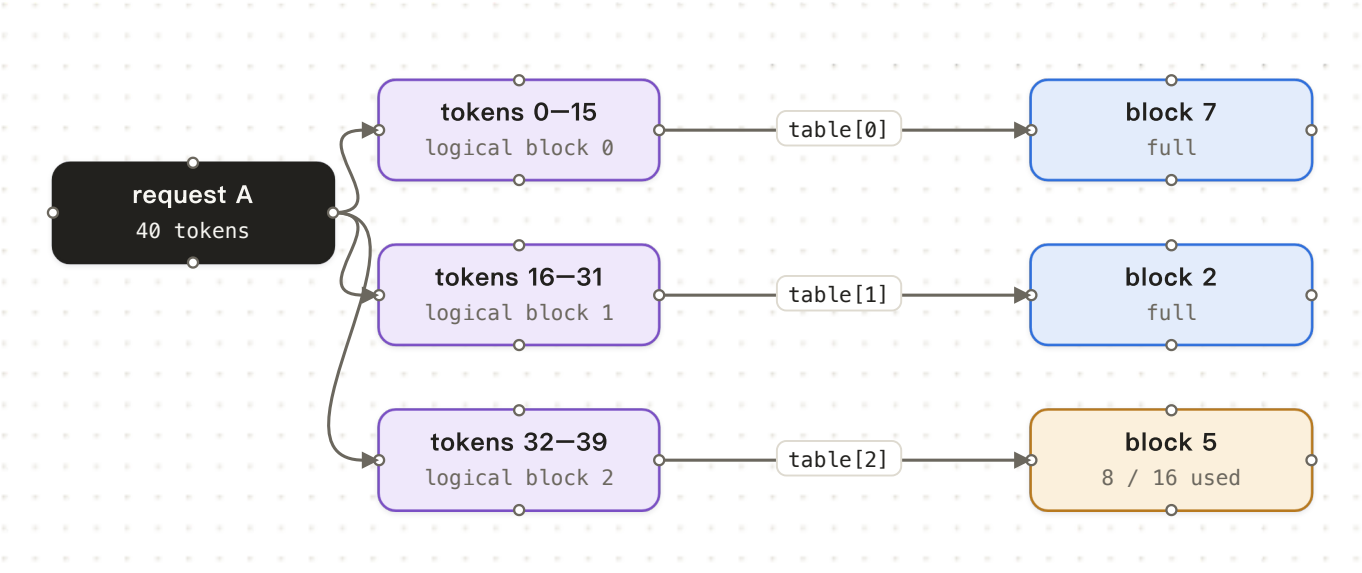
A decoder-only model keeps, for every token it has seen, a key and a value vector per layer: the **KV cache**. For a 8B model in bf16 that is on the order of 128 KB per token, so a single 32k-token conversation needs gigabytes. Three facts shape everything else:

Fact	Consequence
Sequence lengths are unknown up front (the model decides when to stop)	Reserving `max_len` per request wastes most of the memory
Many requests share prefixes (system prompts, few-shot examples, multi-turn chats)	Recomputing and storing the same prefix twice wastes compute and memory
GPU throughput needs large batches, but requests arrive and finish at different times	Batches must be rebuilt every step, not once per batch

The fix: allocate in blocks, like virtual memory

vLLM's PagedAttention splits each sequence's KV cache into fixed-size **blocks** (16 tokens by default). A request owns a list of block ids, its *block table*; blocks of one request need not be contiguous. Memory is allocated one block at a time as the sequence grows, so waste is at most one partially filled block per request.

A block table maps a sequence onto scattered blocks



The attention kernel reads K/V through the block table, so physical blocks can live anywhere in the pool. Only the last block can be partly empty.

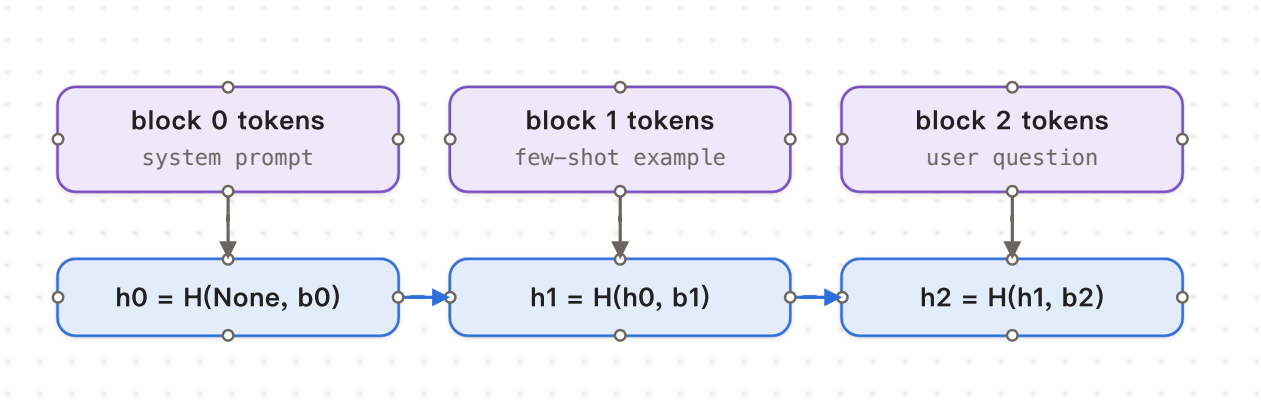
Analogy · A library with fixed-size shelves

Instead of reserving a whole bookcase for every reader "in case they borrow a lot", the library hands out one shelf at a time and keeps a card (the block table) saying which shelves belong to whom. A reader's shelves can be anywhere in the building. When two readers want the same reference books, the librarian points both cards at the same shelf and counts how many cards point there (the reference count).

Prefix caching: blocks as cache entries

Because a full block's content never changes, it can be identified by a hash of its tokens. vLLM chains the hashes, so block *i*'s hash also covers the hash of block *i−1* (`hash_block_tokens`). Two requests share block *i* only if they agree on every token up to the end of that block, which is exactly the condition for the KV values to be identical.

Chained block hashes



The blue arrows are the parent links. Change one token in block 0 and every later hash changes too, so a cache lookup can stop at the first miss.

The pool keeps a `hash → block` map. The important trick is what happens on free (`BlockPool.free_blocks`): a block whose reference count drops to 0 goes to the back of a **free queue** but *keeps its hash*. It is now both free memory and a cache entry. Only when the block is taken from the front of the queue for new content is its hash evicted (`_maybe_evict_cached_block`). A cache hit on a free block simply takes it back out of the queue (`touch`).

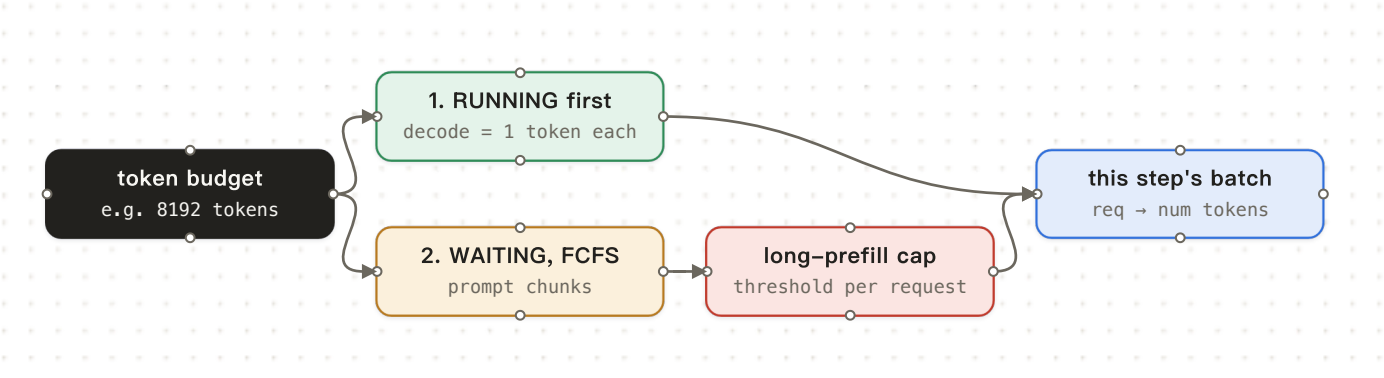
Key point · Free queue = LRU order

The free queue (`FreeKVCacheBlockQueue`) is ordered by eviction priority: front evicted first. Requests free their blocks **in reverse** (`single_type_kv_cache_manager.py`), so a sequence's tail, the part least likely to be shared, is evicted before its head. Drill A1 builds exactly this.

Continuous batching: one token budget per step

Each engine step runs one forward pass over a batch. The v1 scheduler (`Scheduler.schedule`) does not distinguish "prefill" and "decode" requests. Every request has `num_tokens` (prompt + generated) and `num_computed_tokens` (already in the KV cache), and the scheduler hands out up to `max_num_scheduled_tokens` new tokens per step:

One scheduling step



Running requests are served first (`L209`), then waiting ones while budget and `max_num_seqs` allow (`L348`). A long prompt is split across steps: **chunked prefill**.

Why this design:

- **Decodes first** keeps inter-token latency steady: a running request already holds KV memory, and stalling it wastes that memory.
- **Chunked prefill** lets a 30k-token prompt enter without freezing all decoding requests for a whole step; the optional long-prefill threshold caps how much of the step one prompt can take.
- **FCFS without skipping** prevents starvation: if the head of the waiting queue cannot be admitted, smaller requests behind it wait too.

> **Analogy · A kitchen with a fixed number of burners per minute**
>
> Every minute (step) the kitchen has a fixed number of burner-slots (token budget). Dishes already cooking (running requests) need one slot to finish their next stage; they go first so nothing burns. New orders (waiting requests) get the remaining slots in arrival order; a banquet order is cooked in several rounds rather than blocking the kitchen.

Where the drills fit

Drill	Rebuilds	Source
A1 BlockPool	ref counts, LRU free queue, chained-hash prefix cache, all-or-nothing allocation	`block_pool.py`
A2 Scheduler step	token budget, running-first, FCFS admission, chunked prefill, long-prefill cap	`scheduler.py`

A1 · TRACK A · MEMORY AND SCHEDULING · 50 MIN · MEDIUM

A1 · BlockPool: ref counts, LRU free queue, prefix cache

A fixed pool of KV blocks shared by all requests: reference counts decide who owns a block, an LRU free queue decides which block is reused, and chained hashes turn freed blocks into a prefix cache.

Files: `mine/a1_block_pool.py` your code · `tests/test_a1_block_pool.py` tests · `solutions/a1_block_pool.py` solution

The drill in one minute

- **Where it lives**: every engine step, the KV-cache manager asks the pool for blocks for new tokens and returns blocks of finished requests. In vLLM v1 this is `BlockPool`.
- **You implement**: `BlockPool.__init__`, `allocate(token_ids) -> (block_ids, num_cached_tokens)` and `free(block_ids)`. `hash_blocks`, `NoFreeBlocks` and the three read-only accessors are given.
- **The rules that make it subtle**:
    1. A cache hit on a block with `ref_cnt == 0` takes that block **out of the free queue**: it is no longer free capacity.
    2. `allocate` is **all or nothing**: check capacity before touching any state.
    3. Fresh blocks come from the **front** of the queue; a fresh block that still holds an old hash is **evicted** from the cache right then.
    4. `free` decrements in **reverse** order and appends ref-0 blocks to the **back**; their hashes stay cached.

The tests check: fresh allocation order · only full blocks are cached · sharing while running · prefix must match from block 0 · freed blocks still hit · reverse free evicts the tail first · hits on free blocks reduce capacity · failure changes nothing · eviction removes the stale hash · double free raises.

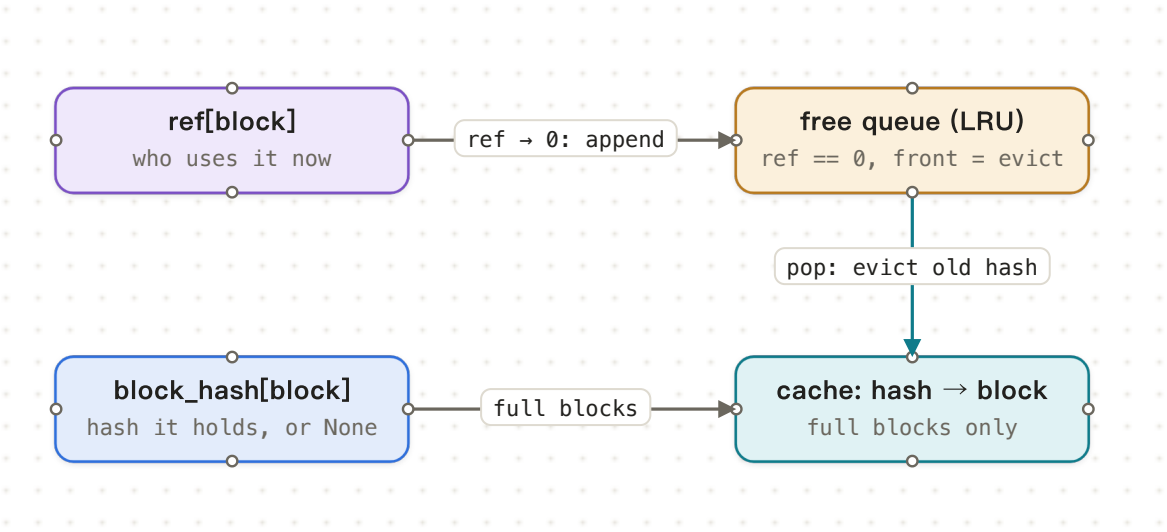
Intuition

Analogy · Hotel rooms with a memory

Each block is a hotel room. `ref_cnt` is how many guests share it. When the last guest checks out, the room goes to the end of the "available" line **but housekeeping does not clean it yet**: if a returning guest asks for "the room with exactly my things in it" (same hash), they get it back instantly. Only when the front desk gives that room to a *new* guest is it cleaned (the old hash evicted). And when a family leaves, the kids' rooms (the tail) are put in line ahead of the parents' room (the head), because the parents' room is the one other relatives are likely to ask for again.

The data structures

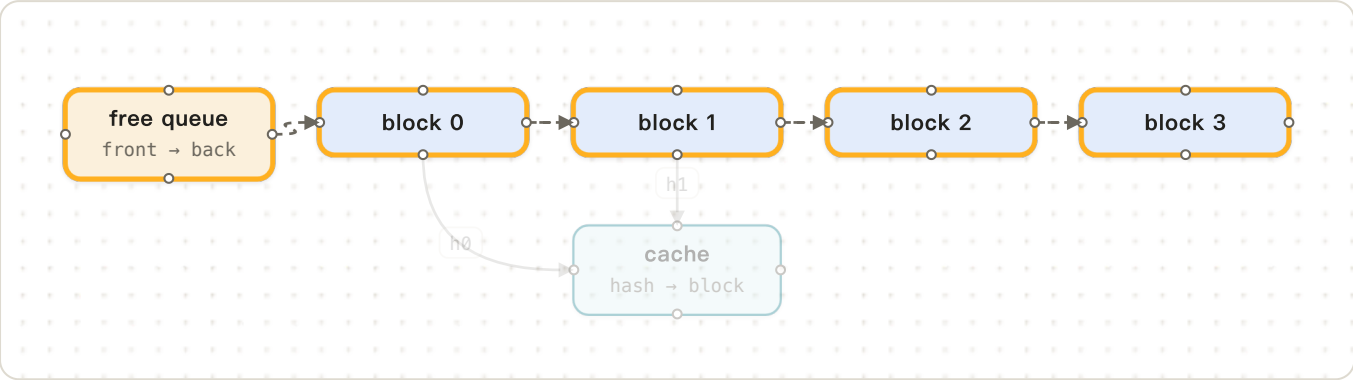
Three structures, one invariant



Invariant: a block is in the free queue **iff** its ref count is 0. The cache may point at free blocks; that is the whole point.

Step through one scenario

▶ **Pool of 4 blocks, block_size 4** 1 / 6



1. Start

All four blocks are free, in id order: queue `0 1 2 3`. Cache empty. **0** **1** **2** **3**

2. allocate(A = 10 tokens)

`hash_blocks` gives 2 hashes (two full blocks; the last 2 tokens are a partial block). No hits. Need ⌈10/4⌉ = 3 blocks → pop `0 1 2` from the front. Register `h0 → 0`, `h1 → 1`; block 2 gets no hash. Queue: `3`. 0 1 2 3

3. allocate(B = A's first 8 tokens + 1 new)

Hashes of B: `h0, h1` → both hit (blocks 0, 1, ref becomes 2). Need 3 − 2 = 1 fresh block → pop `3`. B = `[0, 1, 3]`, `num_cached_tokens = 8`. Queue empty.

4. free(A)

Reverse order: block 2 → ref 0 → append; block 1 → ref 1; block 0 → ref 1. Queue: `2`. Blocks 0 and 1 are still used by B.

5. free(B)

Reverse: 3 → 0 (append), 1 → 0 (append), 0 → 0 (append). Queue: `2 3 1 0`. **Tail first**: the shared head (block 0) is the last to be evicted. Cache still holds `h0 → 0`, `h1 → 1`. 2 3 1 0

6. allocate(A again)

Hits `h0 → 0`, `h1 → 1`, both have ref 0, so both leave the queue (capacity drops by 2). Need 1 fresh → pop `2` (front). Queue: `3`. Prefix caching saved 8 tokens of prefill.

Pitfalls

- **Counting hits on free blocks as free capacity.** With 3 free blocks of which 2 are hits, a request that needs 3 fresh blocks must fail. A naive check passes and then crashes popping an empty queue, leaving refs incremented.
- **Mutating before the capacity check.** Touching hits first and raising later leaks references: those blocks can never be freed.
- **Forgetting to evict the stale hash** when a block is reused: the cache then maps an old hash to a block with new content, and a later "hit" returns wrong KV values silently.
- **Evicting the wrong entry**: only delete `cache[old]` if it still points at *this* block.
- **Caching the partial last block**: its content changes as tokens are appended.
- **Freeing in forward order**: correct refcounts, wrong eviction order. The test with a 3–block pool catches it.

Hints

L1 **I/O examples and the key questions** direction (light spoiler)

```
pool = BlockPool(num_blocks=4, block_size=4)
pool.allocate(list(range(10)))        # -> ([0, 1, 2], 0)
pool.allocate(list(range(8)) + [99])  # -> ([0, 1, 3], 8)    blocks 0,1 shared, ref 2
pool.num_free_blocks                  # -> 0
pool.allocate([7])                    # raises NoFreeBlocks, nothing changes
```

Key questions

1. Which data structure gives O(1) "pop the oldest", "append newest" **and** "remove this one from the middle"?
2. A cache hit on a block whose `ref_cnt` is 0: what has to happen to the free queue?
3. How many blocks does a 10-token sequence need with `block_size = 4`, and how many of them get a hash?
4. When exactly is a hash removed from the cache?
5. Why reverse order in `free`?

L2 **Approach and common traps** breakdown & pitfalls (medium spoiler)

- **State**: `ref: list[int]`, `block_hash: list[int | None]`, `free_q: OrderedDict[int, None]` (front = evict first), `cache: dict[int, int]`.
- **allocate**:
    1. `hashes = hash_blocks(...)`, `n = ceil(len / block_size)`.
    2. Walk `hashes` while `cache` has them → `hits`. Stop at the first miss.
    3. `need = n - len(hits)`; `evictable = len(free_q) - (#hits with ref 0)`; raise if `evictable < need`.
    4. Touch hits (remove from `free_q` if ref was 0; `ref += 1`).
    5. Pop `need` blocks from the front; evict their old hash if `cache[old] == block`; set ref 1.
    6. Give hashes to the new full blocks (`setdefault`, first writer wins).
- **free**: `for b in reversed(ids)`: decrement; at 0 append to the back. Raise on ref $\leqslant$ 0.

L3 **Detailed answers** nearly the answer (heavy spoiler)

▼ Answers to the key questions

1. An `OrderedDict` keyed by block id: `popitem(last=False)`, `d[b] = None`, `del d[b]` are all O(1). vLLM uses an intrusive doubly–linked list on the block objects for the same complexity without hashing (`FreeKVCacheBlockQueue`).
2. It must be removed from the free queue (it is in use again), which is why it does not count as free capacity for the same allocation.
3. ⌈10/4⌉ = 3 blocks; only the 2 full blocks get a hash.
4. When the block is popped from the free queue to hold *new* content (and only if the cache entry still points at that block). Freeing never removes hashes.
5. The free queue is the eviction order. Appending tail blocks first puts them nearer the front, so they are evicted before the head blocks, which are the ones other requests are most likely to share.

Reference solution

solutions/a1_block_pool.py finish and test yours first

solutions/a1_block_pool.py 130 lines

```
"""A1 · BlockPool with prefix caching — reference solution.

A fixed pool of KV-cache blocks shared by all requests (vLLM v1 `BlockPool` +
`FreeKVCacheBlockQueue`, simplified to one KV-cache group):

- Every block has a reference count. ref_cnt > 0: some running request uses it.
  ref_cnt == 0: it sits in the free queue and may be handed out again.
- A FULL block (block_size tokens) gets a content hash chained to the hash of the b
lock
  before it, so equal hashes mean "same tokens AND same prefix". Freed blocks keep
their hash:
  until a block is actually reused for something else it is a prefix-cache entry.
- allocate(token_ids) reuses the longest cached prefix, then takes fresh blocks fro
m the
  FRONT of the free queue (least recently freed first). A fresh block that still ca
rries an old
  hash is evicted from the cache at that moment.
- free(block_ids) decrements in REVERSE order and appends blocks that reach 0 to th
e BACK of the
  queue, so a sequence's tail blocks are evicted before its head blocks: heads are
shared more.

API
    hash_blocks(token_ids, block_size) -> list[int]       given: one chained hash p
er full block
    NoFreeBlocks(Exception)                                 given
    BlockPool(num_blocks, block_size)
        .allocate(token_ids) -> (block_ids, num_cached_tokens)   all or nothing
        .free(block_ids) -> None
        .num_free_blocks -> int                            blocks with ref_cnt == 0
        .ref_count(block_id) -> int
        .cached_block(block_hash) -> int | None            block currently register
ed for a hash
"""
from __future__ import annotations

from collections import OrderedDict


class NoFreeBlocks(Exception):
    """Not enough evictable blocks for an allocation. Raised before anything is mod
ified."""
```

```
def hash_blocks(token_ids: list[int], block_size: int) -> list[int]:
    """One hash per FULL block; block i's hash covers its tokens and the hash of bl
ock i-1.

    Chaining is what makes the cache a *prefix* cache: block 3 of two prompts only
matches if
    blocks 0..2 matched too, because their hashes are inside block 3's hash.
    (Python's hash of a tuple of ints is deterministic, which is all a single proce
ss needs;
    vLLM uses sha256 or a seeded builtin hash for the same reason.)
    """
    hashes, parent = [], None
    for start in range(0, len(token_ids) - block_size + 1, block_size):
        parent = hash((parent, tuple(token_ids[start:start + block_size])))
        hashes.append(parent)
    return hashes


class BlockPool:
    def __init__(self, num_blocks: int, block_size: int) -> None:
        self.block_size = block_size
        self.ref = [0] * num_blocks                       # ref_cnt per block id
        self.block_hash: list[int | None] = [None] * num_blocks   # hash a block cu
rrently holds
        # Free queue in eviction order: front = evict first. An OrderedDict gives O
(1)
        # popleft (popitem(last=False)), append, and O(1) removal from the MIDDLE,
which a cache
        # hit on a free block needs. vLLM gets the same with an intrusive doubly-li
nked list
        # (prev/next pointers stored on the block objects) to avoid hashing on the
hot path.
        self.free_q: OrderedDict[int, None] = OrderedDict((b, None) for b in range
(num_blocks))
        self.cache: dict[int, int] = {}                   # block hash -> block id

    @property
    def num_free_blocks(self) -> int:
        return len(self.free_q)

    def ref_count(self, block_id: int) -> int:
        return self.ref[block_id]

    def cached_block(self, block_hash: int) -> int | None:
        return self.cache.get(block_hash)

    def allocate(self, token_ids: list[int]) -> tuple[list[int], int]:
        hashes = hash_blocks(token_ids, self.block_size)
        n_blocks = -(-len(token_ids) // self.block_size)     # ceil: a partial last
block still needs a block
```

```
        # 1. Longest cached prefix. Stop at the FIRST miss: thanks to chaining, a later hit after a
        #    miss is impossible for honest hashes, and stopping keeps the blocks contiguous.
        hits: list[int] = []
        for h in hashes:
            b = self.cache.get(h)
            if b is None:
                break
            hits.append(b)
        need = n_blocks - len(hits)

        # 2. Capacity check BEFORE mutating anything (all-or-nothing). A hit on a block whose
        #    ref_cnt is 0 takes that block out of the free queue, so it no longer counts as free.
        hits_in_free = sum(1 for b in hits if self.ref[b] == 0)
        if self.num_free_blocks - hits_in_free < need:
            raise NoFreeBlocks(f"need {need} blocks, {self.num_free_blocks - hits_in_free} evictable")

        # 3. Touch the hits: ref 0 -> 1 removes the block from the eviction candidates.
        for b in hits:
            if self.ref[b] == 0:
                del self.free_q[b]
            self.ref[b] += 1

        # 4. Fresh blocks from the front of the queue (least recently freed). A block that still
        #    holds an old hash is evicted from the cache now, and only if the cache entry is still
        #    this block (another block may have been registered for that hash since).
        fresh: list[int] = []
        for _ in range(need):
            b, _ = self.free_q.popitem(last=False)
            old = self.block_hash[b]
            if old is not None and self.cache.get(old) == b:
                del self.cache[old]
            self.block_hash[b] = None
            self.ref[b] = 1
            fresh.append(b)

        # 5. Register the new FULL blocks in the cache. The partial last block gets no hash: its
        #    content will still change as tokens are appended. If the hash is already cached (two
        #    requests computed the same block), keep the existing entry: the first one wins.
        blocks = hits + fresh
        for i in range(len(hits), len(hashes)):
```

```
            h = hashes[i]
            self.block_hash[blocks[i]] = h
            self.cache.setdefault(h, blocks[i])
        return blocks, len(hits) * self.block_size

    def free(self, block_ids: list[int]) -> None:
        # Reverse order: the tail of a sequence is the least likely to be shared, s
o it should be
        # evicted first, i.e. sit nearer the FRONT of the queue. Appending tail-fir
st does that.
        for b in reversed(block_ids):
            if self.ref[b] <= 0:
                raise ValueError(f"block {b} freed more times than allocated")
            self.ref[b] -= 1
            if self.ref[b] == 0:
                self.free_q[b] = None    # back of the queue; the hash (if any) sta
ys cached
```

Follow-ups and the real source

- vLLM keeps one more rule this drill skips: it never serves the *whole* prompt from cache, it recomputes at least the last token so the model produces logits for sampling. Where would you add that?
- How does the pool change when a model has several KV-cache groups (e.g. sliding-window + full attention layers)? Look at `kv_cache_coordinator.py`.
- Compare with the real code: `get_new_blocks` · `touch` · `free_blocks` · `cache_full_blocks`.

A2 · TRACK A · MEMORY AND SCHEDULING · 45 MIN · MEDIUM

A2 · Scheduler step: token budget, chunked prefill, FCFS admission

One scheduling step of a continuous-batching engine: hand out a fixed token budget, running requests first, then new prompts in arrival order, splitting long prompts into chunks.

Files: `mine/a2_token_budget_scheduler.py` your code · `tests/test_a2_token_budget_scheduler.py` tests · `solutions/a2_token_budget_scheduler.py` solution

The drill in one minute

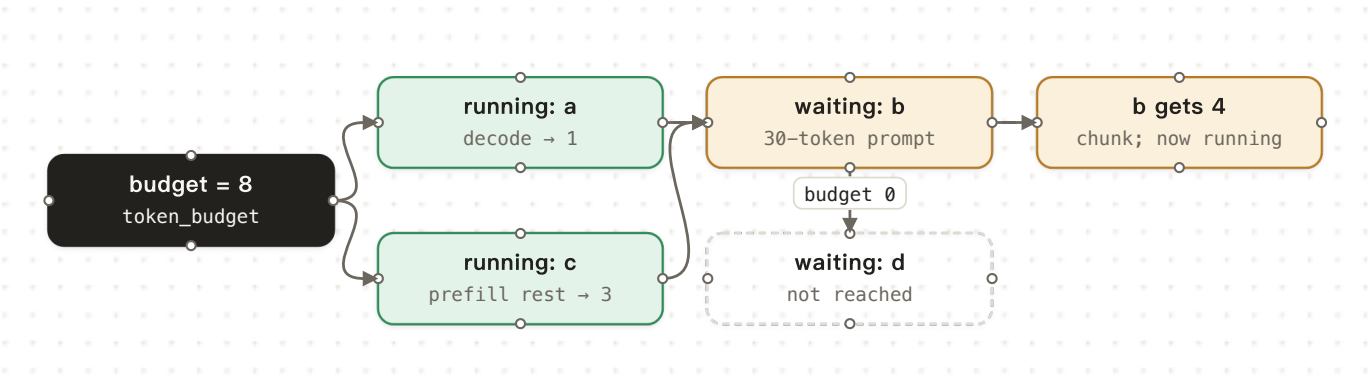
- **Where it lives**: the engine calls `schedule()` once per forward pass; the result is the batch. In vLLM v1 this is `Scheduler.schedule`.
- **You implement**: `Scheduler.__init__`, `add`, `schedule() -> {req_id: num_tokens}` and `update(scheduled) -> finished ids`. `Request` is given.
- **One formula drives everything**: a request needs `num_tokens − num_computed_tokens` more tokens, where `num_tokens = prompt + generated`. That is the rest of the prompt while prefilling and exactly 1 while decoding.
- **Rules**: running first, in order · then waiting FCFS while budget > 0 and `len(running) < max_num_seqs` · each request gets `min(need, threshold, budget)` · a request becomes running with its first chunk · never skip the head of the waiting queue.
- **update**: add the scheduled tokens to `num_computed_tokens`; if that caught up with `num_tokens`, one token is generated; at `max_new_tokens` the request finishes and leaves `running`.

Intuition

Analogy · Burner slots in a kitchen

Every minute the kitchen has 8 burner–slots. Dishes already on the stove each need one slot to cook their next bite, so they are served first. New orders get what is left, in the order they came in. A banquet order that needs 30 slots is cooked over several minutes instead of blocking the kitchen. And the kitchen never lets a small order jump ahead of a waiting banquet, or the banquet would wait forever.

How the budget is spent in one step



Running first: `a` 1 + `c` 3 = 4, leaving 4 for `b`, the head of the waiting queue. 1 + 3 + 4 = 8, so the budget is gone and `d` waits for the next step. Even if budget were left, `d` could never be admitted ahead of `b`: FCFS.

Step through a small run

▶ **budget 8, max_num_seqs 2, two requests** 1 / 4

1. Step 1: a arrives (prompt 5, wants 3 tokens)

`a` needs 5 − 0 = 5 → gets 5. `schedule() == {"a": 5}`. After `update`, computed = 5 = num_tokens → `a` produces **token 1** (num_tokens becomes 6).

2. Step 2: b arrives (prompt 20, wants 1 token)

Running first: `a` needs 6 − 5 = 1 → 1 → **token 2**. Budget left 7 → `b` gets min(20, 7) = 7 and becomes running. `{"a": 1, "b": 7}`. `b` produces nothing yet: 7 < 20.

3. Step 3: a finishes

`a` → 1 → **token 3** = max_new_tokens → **finished**, leaves running. `b` needs 20 − 7 = 13 → gets the remaining 7 (computed 14). `update` returns `["a"]`.

4. Step 4: b finishes

Only `b` is running: needs 6 → gets 6 → computed 20 = num_tokens → first token → finished (max_new_tokens = 1). `update` returns `["b"]`; step 5 schedules nothing.

Pitfalls

- **Treating decode and prefill as different code paths.** One formula covers both; separate paths usually double-count or forget the token produced by the last prefill chunk.
- **Generating a token after a partial chunk.** Only a pass that reaches `num_tokens` produces a token.
- **Skipping a blocked waiting request** to admit a smaller one: passes a quick test, fails the FCFS test, and starves big prompts in production.
- **Checking `max_num_seqs` against waiting instead of running**, or after appending.
- **Iterating over `running` while removing from it** in `update`: iterate over a copy.
- **Forgetting the threshold** applies to running prefills too, not only new ones.

Hints

L1 **I/O examples and the key questions** direction (light spoiler)

```
s = Scheduler(token_budget=8, max_num_seqs=4)
s.add(Request("a", num_prompt_tokens=20, max_new_tokens=2))
s.schedule()          # {"a": 8}
s.update({"a": 8})    # []        no token yet: 8 < 20
s.schedule()          # {"a": 8}
s.update({"a": 8}); s.schedule()   # {"a": 4}
```

Key questions

1. What is `need` for a request that is decoding? For one halfway through its prompt?
2. Why serve running requests before waiting ones?
3. When the head of the waiting queue gets 0 tokens, do you try the next one?
4. In `update`, how do you know whether this step produced a token for a request?

L2 **Approach and common traps** breakdown & pitfalls (medium spoiler)

- `need = req.num_tokens - req.num_computed_tokens`; cap with the threshold if `0 < threshold < need`; then `min(need, budget)`.
- `schedule`: loop over `running` while budget > 0; then `while waiting and budget > 0 and len(running) < max_num_seqs`: popleft, compute n, append to running, record.
- `update`: for each running request with `n > 0`: `num_computed_tokens += n`; if it equals `num_tokens`, `num_output_tokens += 1`; at `max_new_tokens` remove and report. Iterate over `list(self.running)`.

L3 **Detailed answers** nearly the answer (heavy spoiler)

▼ Answers to the key questions

1. Decoding: exactly 1 (the token generated in the previous step has no KV yet). Halfway: the rest of the prompt, possibly capped by the threshold and the budget.
2. They already hold KV blocks and their users are waiting for the next token; starving them wastes memory and makes inter-token latency spiky.
3. No. With `budget > 0` the head always gets at least 1 token, so the loop only stops on budget 0 or `max_num_seqs`. Stopping instead of skipping is what makes the queue FCFS.
4. Compare after adding: `num_computed_tokens == num_tokens` means the pass covered the last known token, so its logits are sampled.

Reference solution

solutions/a2_token_budget_scheduler.py finish and test yours first

solutions/a2_token_budget_scheduler.py 104 lines

```
"""A2 · One step of a continuous-batching scheduler with chunked prefill — referenc
e solution.

The vLLM v1 scheduler does not have separate "prefill" and "decode" phases. Every r
equest just has
`num_tokens` (prompt + generated so far) and `num_computed_tokens` (how many of tho
se already have
KV in the cache); each step it hands out at most `token_budget` new tokens:

1. RUNNING requests first, in order. Each gets min(remaining, long_prefill_threshol
d, budget)
   tokens, where remaining = num_tokens - num_computed_tokens (1 for a decoding req
uest).
2. Then WAITING requests, first come first served, while budget is left and fewer t
han
   max_num_seqs requests are running. A prompt bigger than the budget is split acro
ss steps
   (chunked prefill); the request becomes RUNNING as soon as it gets its first chun
k.

`update(scheduled)` plays the model: it adds the scheduled tokens to num_computed_t
okens, and a
request whose KV caught up with all its tokens samples one new token. A request tha
t reaches
max_new_tokens finishes and leaves the running list.

API
    Request(req_id, num_prompt_tokens, max_new_tokens)               given (datacla
ss)
    Scheduler(token_budget, max_num_seqs, long_prefill_threshold=0)
        .add(req)                       append to the waiting queue
        .schedule() -> dict[str, int]   req_id -> tokens to run this step (insertion
order = batch order)
        .update(scheduled) -> list[str]   apply one model step, return the finished
req_ids
        .waiting (deque) / .running (list)
"""
from __future__ import annotations

from collections import deque
from dataclasses import dataclass
```

```python
@dataclass
class Request:
    req_id: str
    num_prompt_tokens: int
    max_new_tokens: int
    num_computed_tokens: int = 0    # tokens whose KV is already in the cache
    num_output_tokens: int = 0      # tokens generated so far

    @property
    def num_tokens(self) -> int:
        """All tokens of the sequence so far: prompt + generated."""
        return self.num_prompt_tokens + self.num_output_tokens


class Scheduler:
    def __init__(self, token_budget: int, max_num_seqs: int, long_prefill_threshol
d: int = 0) -> None:
        self.token_budget = token_budget
        self.max_num_seqs = max_num_seqs
        # 0 = no cap. A cap stops one huge prompt from taking the whole step and st
alling every
        # decoding request behind it (vLLM: scheduler_config.long_prefill_token_thr
eshold).
        self.long_prefill_threshold = long_prefill_threshold
        self.waiting: deque[Request] = deque()
        self.running: list[Request] = []

    def add(self, req: Request) -> None:
        self.waiting.append(req)

    def _tokens_for(self, req: Request, budget: int) -> int:
        n = req.num_tokens - req.num_computed_tokens          # 1 while decoding, t
he rest of the prompt while prefilling
        if 0 < self.long_prefill_threshold < n:
            n = self.long_prefill_threshold
        return min(n, budget)

    def schedule(self) -> dict[str, int]:
        budget = self.token_budget
        out: dict[str, int] = {}

        # 1. RUNNING first: they already hold KV blocks, and starving them wastes t
hat memory and
        #    hurts inter-token latency. Order matters only when the budget runs out
mid-list.
        for req in self.running:
            if budget == 0:
                break
            n = self._tokens_for(req, budget)
            if n > 0:
                out[req.req_id] = n
                budget -= n
```

```
        # 2. WAITING, FCFS. Stop (don't skip) when a request cannot be admitted: sk
ipping would let
        #    later small requests overtake an earlier big one forever (starvation).
        while self.waiting and budget > 0 and len(self.running) < self.max_num_seq
s:
            req = self.waiting.popleft()
            n = self._tokens_for(req, budget)
            self.running.append(req)        # RUNNING from its first chunk on
            out[req.req_id] = n
            budget -= n

        assert budget >= 0 and len(self.running) <= self.max_num_seqs
        return out

    def update(self, scheduled: dict[str, int]) -> list[str]:
        finished: list[str] = []
        for req in list(self.running):
            n = scheduled.get(req.req_id, 0)
            if n == 0:
                continue
            req.num_computed_tokens += n
            # A forward pass only yields a usable next token when it covered the LA
ST known token:
            # a partial prefill chunk produces logits nobody samples from.
            if req.num_computed_tokens == req.num_tokens:
                req.num_output_tokens += 1
                if req.num_output_tokens >= req.max_new_tokens:
                    self.running.remove(req)
                    finished.append(req.req_id)
        return finished
```

Follow-ups and the real source

- Real schedulers must also get KV blocks for every scheduled token (drill A1). What should happen when a running request cannot get a block? Read how vLLM preempts the lowest-priority running request in `schedule`.
- With prefix caching, a new request may start with `num_computed_tokens > 0`. Which line of your code changes?
- The real loops: running first at `L209–L221`, waiting at `L348–L351`, the long-prefill cap at `L219`.